

■ Homework #2 2025학년 2학기

교과목 / 담당교수	운영체제 / 김남규	주차	12 주차
이름	권영훈	학번	20213032

■ 객관식 문제 (각 2점)

번호	2	6	12	19	20
페이지	344	344	345	346	346
답	1	4	2	1	1, 3

페이지	문제
344	2. 멀티스레드가 동시에 공유 데이터를 접근할 때 공유 데이터가 훼손되지 않도록 막는 기법의 핵심 정책은 무엇인가? ① 상호배제 ② 컨텍스트 스위칭 ③ 인터럽트 금지 ④ 시스템 호출
344	6. 다음 원자명령에 대한 설명으로 옳은 것은? TSL ax, lock ① ax 값을 lock 변수에 기록하고 ax에 0을 저장한다. ② ax 값을 lock 변수에 기록하고 ax에 1을 저장한다. ③ lock 변수 값을 ax에 읽어오고 lock 변수에 0을 저장한다. ④ lock 변수 값을 ax에 읽어오고 lock 변수에 1을 저장한다.
345	12. 커널 코드나 인터럽트 서비스 루틴 등에서 임계구역에 접근할 때 멀티스레드 동기화를 위해 사용되는 기법은? ① 뮤텍스락 ② 스핀락 ③ 세마포 ④ 커널에서는 스레드 동기화가 필요 없다.
346	19. 생산자 소비자 문제에 대해 잘 설명한 것은? ① 공유 버퍼를 사이에 두고 공유 버퍼에 데이터를 저장하는 스레드와 공유 버퍼로부터 데이터를 읽어가는 스레드 사이에 공유 버퍼에 대한 동시 접근 문제이다. ② 생산자 소비자 문제는 스레드 동기화 문제라고 볼 수 없다. ③ 생산자보다는 소비자에게 항상 문제가 발생한다. ④ 생산자 소비자 문제는 단일 스레드로 작성되는 응용프로그램에서 주로 발생한다.
346	20. 생산자 소비자 문제인 것을 모두 골라라. ① 마이크로부터 연속하여 들어오는 오디오 샘플을 큐에 넣는 스레드와 큐의 오디오 샘플을 스피커로 출력하는 스레드 ② a.c 파일을 찾기 위해 디스크 드라이버 C와 D를 각각 검색하는 2개의 스레드 ③ 프린트할 파일이 생길 때마다 큐에 넣은 스레드와 큐에 있는 파일을 프린터로 출력하는 스레드 ④ 1에서 10000까지 더하기 위해 1에서 5000까지 더하는 스레드와 5001에서 10000까지 더하는 스레드, 그리고 이들이 모두 더하기를 기다려 두 결과를 합하여 화면에 출력하는 스레드

■ 복합 문제 (10점, 350페이지)

6. 본문의 탐구 6-5를 참고하여 다음 설명과 같이 작동하는 응용프로그램을 작성하라.

큐를 만들고, 랜덤한 시간 간격으로 큐에 정수를 저장하는 생산자 스레드와 큐에 있는 정수를 읽어 정수의 개수만큼 화면에 별문자(*)를 출력하는 소비자 스레드를 가진 멀티스레드 응용프로그램 `plotter.c`을 작성하라. 큐는 정수형 배열(`int queue[]`)로 구현하면 된다.

이 응용프로그램을 컴파일하고 실행하는 과정과 실행 결과는 다음과 같다. 실행 결과는, 생산자가 큐에 19, 14, 10, 7, 2, 3, 20, 17, 6, 13 등의 정수를 랜덤하게 계속 삽입한 상황으로, 소비자 스레드가 각 숫자대로 별문자(*)를 출력하고 있는 장면이다.

```
$ gcc -o plotter plotter.c -lpthread
$ ./plotter
*****
*****
*****
*****
**
***
*****
*****
*****
*****
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define BUF_SIZE 5
int queue[BUF_SIZE];
int count = 0;
int in = 0;
int out = 0;

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

void *producer(void *param){
    int item;
    srand(time(NULL));

    while (1){
        pthread_mutex_lock(&mutex);

        while (count == BUF_SIZE){
            printf("Producer: Buffer Full. Waiting...Wn");
            pthread_cond_wait(&not_full,&mutex);
        }

        item = (rand () % 20) + 1;
        queue[in] = item;
        in =(in + 1)%BUF_SIZE;
        count++;
```

```

        printf("Producer: Produced %d. (Count: %d)\n",item,count);

        pthread_cond_signal(&not_empty);

        pthread_mutex_unlock(&mutex);

        usleep((rand() % 1000) * 1000);
    }
    return NULL;
}

void *consumer(void *param){
    int item;

    while (1){
        pthread_mutex_lock(&mutex);

        while (count ==0){
            printf("Consumer: Buffer Empty. Waiting...\n");

            pthread_cond_wait(&not_empty,&mutex);
        }

        item =queue[out];
        out =(out +1)%BUF_SIZE;
        count--;

        pthread_cond_signal(&not_full);

        pthread_mutex_unlock(&mutex);

        printf("Consumer: Plotting %2d: ",item);
        for (int i =0;i <item;i++){
            putchar('*');
        }
        putchar('\n');

        usleep((rand() % 1000) * 1000);
    }
    return NULL;
}

int main(){
    pthread_t prod_tid,cons_tid;

    pthread_mutex_init(&mutex,NULL);
    pthread_cond_init(&not_full,NULL);
    pthread_cond_init(&not_empty,NULL);

    pthread_create(&prod_tid,NULL,producer,NULL);
    pthread_create(&cons_tid,NULL,consumer,NULL);

    pthread_join(prod_tid,NULL);
    pthread_join(cons_tid,NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    return 0;
}

```